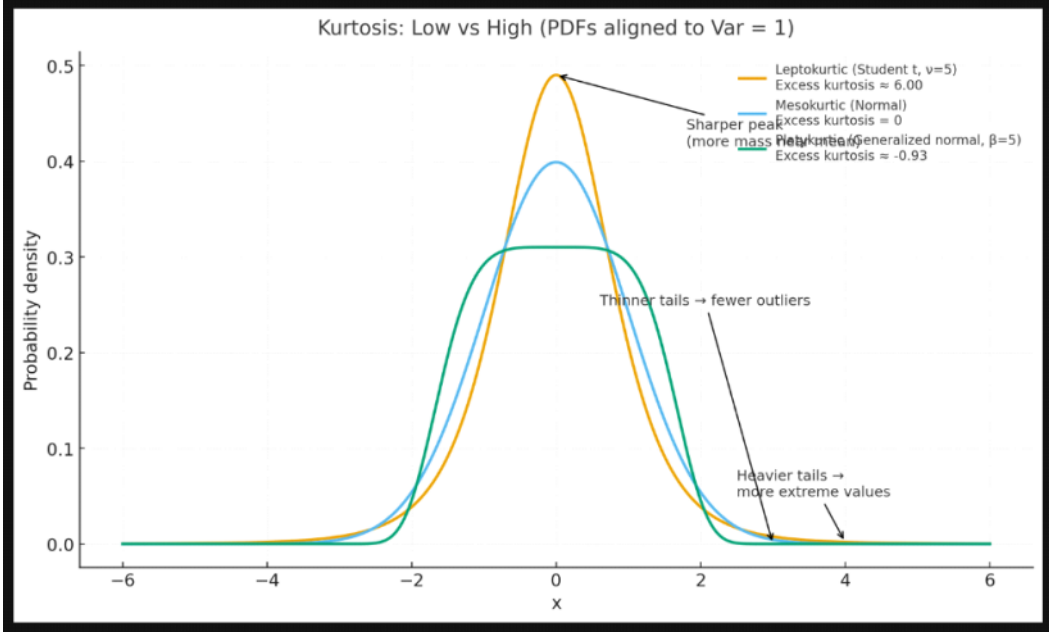
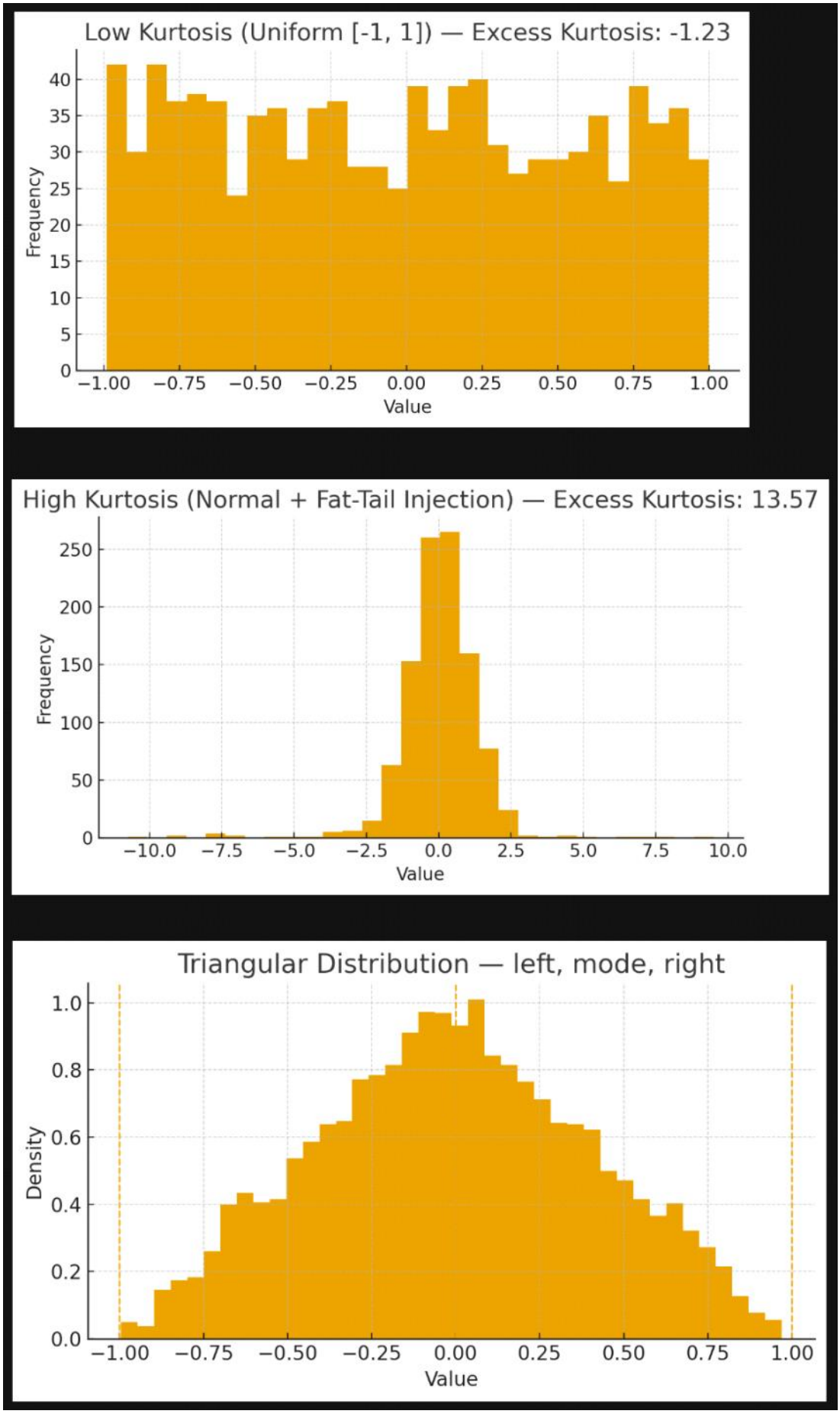
Descriptive Statistics & Probability

🎯 Session Objectives:

- ✅ Understand the concepts of range and interquartile range (IQR) and calculate them from a given dataset. 📏
- ✅ Compute the variance and standard deviation and interpret their significance in data dispersion. 📐
- ✅ Recognise and interpret skewness and kurtosis to understand the shape and distribution of data. 📊
- ✅ Define experiment, sample space, and event clearly.
- ✅ Compute probabilities of simple events from a sample space.
- ✅ Tell apart independent, dependent, and mutually exclusive events (with formulas).
- ✅ Simulate simple and compound events in Python.
- ✅ Compute conditional probability and apply it to real-world cases.
- ✅ Use the Law of Total Probability and Bayes' Theorem to update beliefs with new evidence.

Kurtosis is a statistical measure that describes the **degree of "tailedness"** in the probability distribution of a real-valued random variable. It provides insight into the shape of the distribution, particularly in terms of the presence of outliers. There are different types of kurtosis, such as **leptokurtic** (high kurtosis, indicating heavy tails) and **platykurtic** (low kurtosis, indicating light tails). Additionally, **kurtosis risk** refers to the risk that arises when a statistical model assumes a normal distribution but is applied to data that may have extreme values. Wikipedia +1




Low Kurtosis (Uniform [-1, 1]) — Excess Kurtosis: -1.23
Frequency
Value
High Kurtosis (Normal + Fat-Tail Injection) — Excess Kurtosis: 13.57
Frequency
Value
Triangular Distribution — left, mode, right
Density
Value

```python
# Descriptive Statistics -> 'Kurtosis'
import numpy as np
import pandas as pd
import seaborn as sns
from scipy.stats import kurtosis
```

```python
np.random.seed(42)
portfolio_returns = np.random.normal(loc = 0, scale = 1, size = 1000)
portfolio_returns
```

```
array([ 4.96714153e-01, -1.38264301e-01,  6.47688538e-01,  1.52302986e+00,
       -2.34153375e-01, -2.34136957e-01,  1.57921282e+00,  7.67434729e-01,
       -4.69474386e-01,  5.42560044e-01, -4.63417693e-01, -4.65729754e-01,
        2.41962272e-01, -1.91328024e+00, -1.72491783e+00, -5.62287529e-01,
       -1.01283112e+00,  3.14247333e-01, -9.08024076e-01, -1.41230370e+00,
        1.46564877e+00, -2.25776300e-01,  6.75282047e-02, -1.42474819e+00,
       -5.44382725e-01,  1.10922590e-01, -1.15099358e+00,  3.75698018e-01,
       -6.00638690e-01, -2.91693750e-01, -6.01706612e-01,  1.85227818e+00,
       -1.34972247e-02, -1.05771093e+00,  8.22544912e-01, -1.22084365e+00,
        2.08863595e-01, -1.95967012e+00, -1.32818605e+00,  1.96861236e-01,
        7.38466580e-01,  1.71368281e-01, -1.15648282e-01, -3.01103696e-01,
       -1.47852199e+00, -7.19844208e-01, -4.60638771e-01,  1.05712223e+00,
        3.43618290e-01, -1.76304016e+00,  3.24083969e-01, -3.85082280e-01,
       -6.76922000e-01,  6.11676289e-01,  1.03099952e+00,  9.31280119e-01,
       -8.39217523e-01, -3.09212376e-01,  3.31263431e-01,  9.75545127e-01,
       -4.79174238e-01, -1.85658977e-01, -1.10633497e+00, -1.19620662e+00,
        8.12525822e-01,  1.35624003e+00, -7.20101216e-02,  1.00353290e+00,
```

```python
portfolio_returns.shape
```

```
(1000,)
```

```python
kurtosis_value = kurtosis(portfolio_returns)
kurtosis_value
```

```
0.06620589292148393
```

```python
portfolio_returns = np.append(portfolio_returns , np.random.normal(loc = 0, scale = 5, size = 50))
portfolio_returns
```

```
array([ 0.49671415, -0.1382643 ,  0.64768854, ..., -7.88196078,
        7.38270175,  6.90045677])
```

```python
portfolio_returns.shape
```

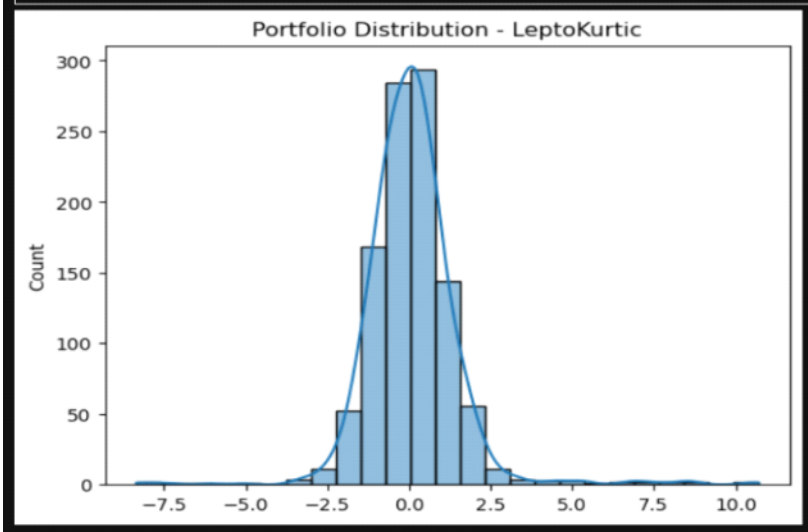
```
(1050,)
```

```python
kurtosis_value = kurtosis(portfolio_returns)
kurtosis_value
```

```
13.129132194044388
```

```python
# Visualize
import matplotlib.pyplot as plt
sns.histplot(portfolio_returns , bins = 25 , kde = True)
plt.title("Portfolio Distribution - LeptoKurtic")
plt.show()
```

```
plt.show()
```



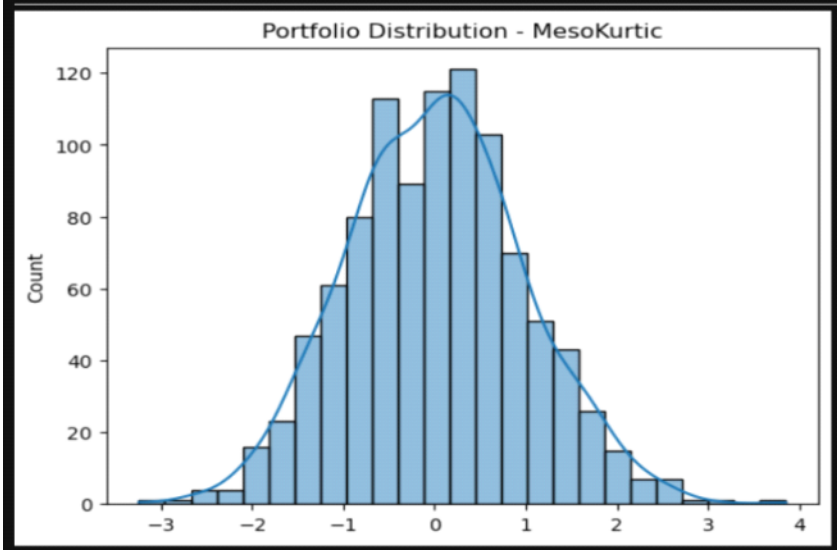

```
np.random.seed(42)
portfolio_returns = np.random.normal(loc = 0, scale = 1, size = 1000)
portfolio_returns
```

```
array([ 4.96714153e-01, -1.38264301e-01,  6.47688538e-01,  1.52302986e+00,
       -2.34153375e-01, -2.34136957e-01,  1.57921282e+00,  7.67434729e-01,
       -4.69474386e-01,  5.42560044e-01, -4.63417693e-01, -4.65729754e-01,
        2.41962272e-01, -1.91328024e+00, -1.72491783e+00, -5.62287529e-01,
       -1.01283112e+00,  3.14247333e-01, -9.08024076e-01, -1.41230370e+00,
        1.46564877e+00, -2.25776300e-01,  6.75282047e-02, -1.42474819e+00,
       -5.44382725e-01,  1.10922590e-01, -1.15099358e+00,  3.75698018e-01,
       -6.00638690e-01, -2.91693750e-01, -6.01706612e-01,  1.85227818e+00,
       -1.34972247e-02, -1.05771093e+00,  8.22544912e-01, -1.22084365e+00,
        2.08863595e-01, -1.95967012e+00, -1.32818605e+00,  1.96861236e-01,
        7.38466580e-01,  1.71368281e-01, -1.15648282e-01, -3.01103696e-01,
       -1.47852199e+00, -7.19844208e-01, -4.60638771e-01,  1.05712223e+00,
        3.43618290e-01, -1.76304016e+00,  3.24083969e-01, -3.85082280e-01,
       -6.76922000e-01,  6.11676289e-01,  1.03099952e+00,  9.31280119e-01,
       -8.39217523e-01, -3.09212376e-01,  3.31263431e-01,  9.75545127e-01,
       -4.79174238e-01, -1.85658977e-01, -1.10633497e+00, -1.19620662e+00,
        8.12525822e-01,  1.35624003e+00, -7.20101216e-02,  1.00353290e+00,
```

```
kurtosis_value = kurtosis(portfolio_returns)
kurtosis_value
```

```
0.06620589292148393
```

```
# Visualize
sns.histplot(portfolio_returns , bins = 25 , kde = True)
plt.title("Portfolio Distribution - MesoKurtic")
plt.show()
```



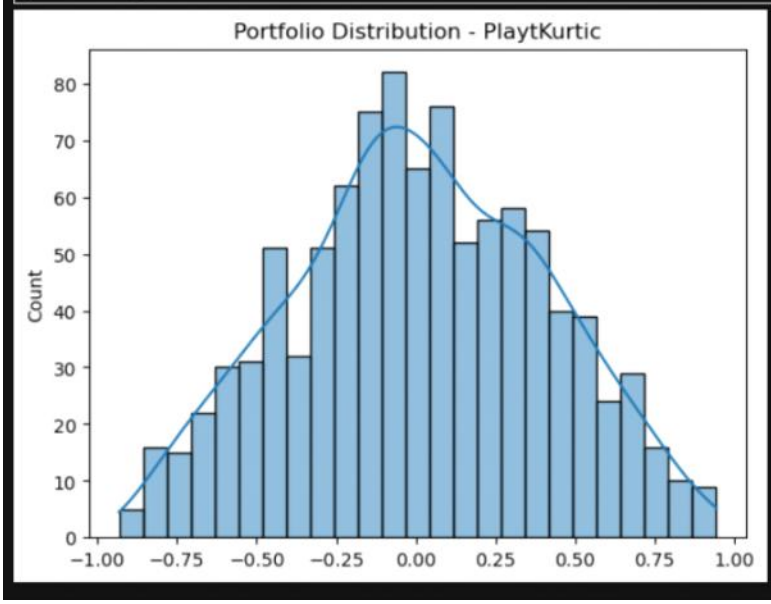
```python
np.random.seed(11)
portfolio_returns = np.random.uniform(low=-1, high=1 , size = 1000)
# Platykurtic [Low Kurtosis]
portfolio_returns = np.random.triangular(left = -1 , mode = 0 ,right = 1, size = 1000)
portfolio_returns
```

```
array([-0.48091297, -0.38229706, -0.10364353, -0.10409789, -0.68744713,
        0.9427542 ,  0.43772802,  0.33461569,  0.22802937, -0.10969011,
        0.18501469,  0.43429516,  0.24974975,  0.5170442 ,  0.33366215,
        0.26773377,  0.57910762,  0.3590672 , -0.18485105,  0.00523147,
       -0.56624548, -0.08449681,  0.15148777, -0.34180359,  0.08000894,
       -0.18358379, -0.31859112, -0.15170864,  0.09918257, -0.03623559,
       -0.05645741,  0.32952596,  0.15422272,  0.08896331, -0.52126454,
       -0.80084385, -0.13656917, -0.50774104,  0.16729843,  0.4630678 ,
        0.27525268, -0.05761067,  0.4284737 ,  0.24499217,  0.02150071,
        0.11143951, -0.39720012, -0.79994031,  0.59853729,  0.74239692,
        0.00899215,  0.08262719,  0.35133664, -0.55434271,  0.25298618,
        0.35961434, -0.31056786,  0.06634782,  0.37443552,  0.67634041,
       -0.09121044, -0.44782845,  0.26238599,  0.12151895, -0.0072314 ,
       -0.11541317, -0.76324081,  0.60116425, -0.12615733, -0.69985788,
       -0.5451994 ,  0.09447557, -0.73330882,  0.31304538,  0.28239761,
       -0.90140852,  0.15653802, -0.2172014 , -0.23739371,  0.46378512,
       -0.62186663, -0.08674972,  0.69926291, -0.58377547,  0.11390643,
```

```python
kurtosis_value = kurtosis(portfolio_returns)
kurtosis_value
```

```
-0.5959793906887358
```

```python
# Visualize
sns.histplot(portfolio_returns , bins = 25 , kde = True)
plt.title("Portfolio Distribution - PlaytKurtic")
plt.show()
```




```python
# Descriptive ->
import pandas as pd
process_engineering_data = pd.read_csv('process_engineering_data.csv')
process_engineering_data
```

	process_id	temperature	pressure	flow_rate
0	26bac532-89b5-493d-94cb-3d5dda8f425c	52.483571	127.987109	26.624109
1	08e6adcb-f775-4d17-b129-e24ba29ee93d	49.308678	118.492674	29.277407

0	26bac532-89b5-493d-94cb-3d5dda8f425c	52.483571	127.987109	26.624109
1	08e6adcb-f775-4d17-b129-e24ba29ee93d	49.308678	118.492674	29.277407
2	74d101ae-d735-4a90-b512-c836c7fa7ec5	53.238443	101.192607	26.037900
3	25ac6436-4f7c-4002-80ed-c31906434f38	57.615149	87.061264	28.460192
4	2fdcad4c-cf71-4606-a457-cab5e04fe203	48.829233	113.964466	20.531927
...	...	...	...	...
995	9253debe-9d48-4ef8-a4d5-0d8dfa819b8c	48.594499	121.403005	70.387403
996	0cda244b-b063-4b26-877d-c0d56182c300	58.988433	99.469575	71.288763
997	eb203c5a-24e6-465c-b20a-a00f4a617723	53.204214	82.362507	63.791197
998	0a8d5f6f-e34b-454e-8dc0-e2957f95d753	47.144105	96.738661	71.670882
999	0ed35dd7-b981-4358-bddf-cf91404c8d68	52.862914	85.101947	69.223705

1000 rows × 4 columns

```python
# Exercise 1: Range and IQR
# Range = Max - Min
temp_range = (
    process_engineering_data['temperature'].max() - process_engineering_data['temperature'].min()
)
temp_range
```

```
35.46999415361897
```

```python
process_engineering_data['temperature'].max()
```

```
69.26365745327361
```

```python
# Exercise 1: Range and IQR
# IQR = Q3-Q1
Q3 = process_engineering_data['temperature'].quantile(0.75)
Q1 = process_engineering_data['temperature'].quantile(0.25)
IQR = Q3-Q1
IQR
```

```
6.477670904676408
```

```python
# Exercise 2: Variance and Standard Deviation
# Pressure Variability Analysis
pressure_var = process_engineering_data['pressure'].var()
pressure_var
```

```
397.96609386005645
```

```python
pressure_std = process_engineering_data['pressure'].std()
pressure_std
```

```
19.94908754454841
```

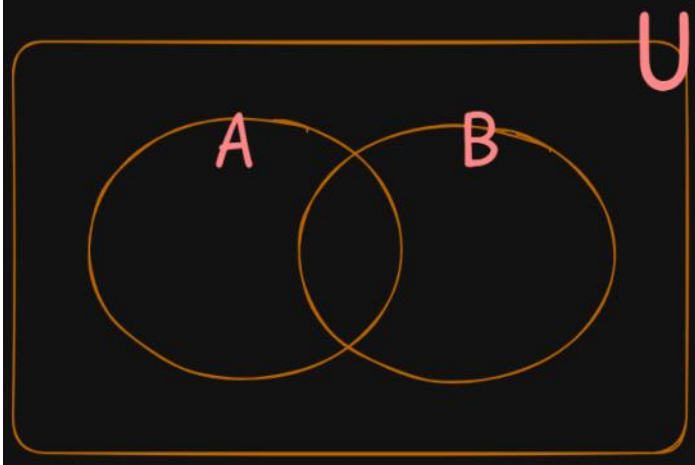
```python
pressure_mean = process_engineering_data['pressure'].mean()
pressure_mean
```

```
101.41672474498311
```

```python
# Exercise 3: Flow Rate Skewness
# Analyzing Distribution Patterns
flow_rate_skew = process_engineering_data['flow_rate'].skew()
flow_rate_skew
```

```
1.331814813830591
```

Probabilities



$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

```python
# Probability -> Rolling a 3 in a six sided dice
import random
sample_space = [1,2,3,4,5,6]
event = [3]
# P(E) = (Number of outcomes favorable to E) / (Total number of outcomes in S)
probability_event = len(event)/len(sample_space) # 1/6
probability_event
```

```
0.16666666666666666
```

```python
1/6
```

```
0.16666666666666666
```

```python
# Drawing an Ace From 52 Deck of Cards....
sample_space = list(range(1,53)) # [1,2,3...52]
print(sample_space)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 3
0, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52]
```

```python
# Position of an Ace -> After every 13th Element
event_ace = [1,14,27,40]
probability_event = len(event_ace)/len(sample_space) # 4/52 ~ 1/13 ~  0.077
probability_event
```

```
0.07692307692307693
```

```python
# Rolling a Dice & Flipping a Coin [Independent Event]
die_rolling = [random.randint(1,6) for _ in range(10000)]
print("Die Rolls: ")
print(die_rolling)
```

```
Die Rolls:
[2, 6, 6, 2, 5, 1, 2, 3, 6, 3, 1, 1, 2, 1, 5, 5, 4, 4, 3, 5, 2, 3, 6, 6, 4, 6, 4, 1, 5, 4, 1, 1, 5, 3, 6, 5,
2, 2, 4, 5, 1, 3, 3, 2, 6, 1, 1, 6, 3, 6, 1, 1, 1, 4, 4, 3, 1, 6, 5, 2, 1, 1, 4, 1, 4, 5, 5, 1, 6, 2, 5, 6,
4, 1, 1, 2, 2, 2, 4, 3, 6, 6, 3, 3, 2, 5, 1, 6, 4, 5, 1, 3, 6, 3, 6, 1, 3, 2, 6, 2, 5, 5, 1, 3, 2, 2, 3, 4,
4, 4, 5, 3, 1, 1, 6, 5, 2, 1, 1, 1, 1, 1, 2, 3, 2, 4, 2, 3, 5, 2, 3, 1, 6, 6, 5, 1, 6, 6, 5, 5, 5, 4, 4, 6,
3, 6, 4, 2, 4, 1, 5, 2, 2, 5, 6, 2, 2, 1, 1, 2, 1, 4, 5, 2, 1, 5, 4, 6, 1, 6, 3, 6, 2, 1, 6, 3, 4, 3, 1, 2,
4, 4, 1, 1, 6, 6, 4, 2, 4, 5, 1, 1, 1, 2, 2, 3, 3, 4, 6, 1, 1, 3, 3, 1, 5, 3, 3, 1, 5, 5, 4, 5, 3, 5, 4, 2,
2, 2, 5, 2, 5, 3, 5, 4, 4, 5, 4, 4, 1, 4, 6, 5, 2, 3, 2, 1, 3, 1, 1, 4, 3, 2, 3, 4, 1, 3, 5, 3, 2, 1, 3, 4,
5, 6, 2, 2, 5, 4, 5, 6, 3, 3, 6, 3, 4, 6, 4, 5, 6, 3, 2, 5, 4, 1, 5, 6, 2, 4, 1, 1, 6, 5, 4, 4, 3, 4, 3, 5,
6, 6, 4, 2, 1, 4, 6, 4, 3, 4, 2, 4, 5, 4, 1, 6, 4, 4, 6, 5, 6, 4, 4, 3, 4, 4, 1, 5, 2, 5, 1, 5, 3, 6, 4, 6,
4, 2, 3, 5, 6, 2, 6, 5, 4, 2, 6, 6, 6, 3, 3, 2, 1, 4, 3, 4, 2, 4, 4, 4, 1, 6, 5, 1, 1, 3, 4, 6, 6, 2, 6, 6,
3, 1, 2, 3, 5, 6, 5, 5, 4, 1, 2, 6, 5, 4, 3, 4, 4, 5, 6, 5, 1, 4, 6, 6, 6, 3, 6, 4, 3, 3, 4, 6, 6, 1, 4, 5,
6, 6, 6, 2, 3, 1, 4, 5, 5, 6, 3, 3, 6, 1, 2, 6, 1, 2, 6, 5, 4, 5, 2, 2, 3, 3, 5, 3, 3, 4, 5, 5, 1, 1, 5, 2,
4, 3, 1, 1, 2, 5, 1, 5, 4, 6, 5, 1, 2, 1, 3, 4, 6, 6, 2, 3, 2, 5, 1, 5, 4, 6, 5, 3, 6, 2, 2, 4, 5, 2, 4, 4,
4, 1, 3, 5, 6, 5, 4, 3, 1, 3, 2, 6, 6, 5, 5, 6, 1, 1, 1, 6, 6, 1, 5, 3, 3, 3, 6, 4, 2, 3, 4, 2, 1, 5, 6, 6,
6, 4, 5, 5, 2, 6, 4, 1, 5, 5, 4, 5, 2, 6, 1, 2, 5, 6, 2, 1, 2, 6, 2, 2, 6, 1, 2, 6, 4, 4, 2, 3, 4, 3, 3, 4,
6, 3, 4, 4, 4, 5, 1, 4, 6, 6, 1, 2, 6, 2, 5, 4, 6, 5, 2, 4, 2, 1, 3, 3, 2, 2, 5, 4, 4, 4, 4, 3, 4, 2, 4, 4,
```

```python
len(die_rolling)
```

```
10000
```

```python
coin_flips = [random.choice(['H','T']) for _ in range(10000)]
print("Flipping a Coin: ")
print(coin_flips)
```

```
Flipping a Coin:
['H', 'H', 'T', 'T', 'H', 'H', 'T', 'H', 'H', 'H', 'T', 'T', 'H', 'H', 'T', 'H', 'T', 'H', 'H', 'H', 'T',
'T', 'T', 'T', 'H', 'T', 'T', 'T', 'H', 'T', 'T', 'T', 'H', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T',
'T', 'T', 'H', 'H', 'H', 'T', 'T', 'T', 'H', 'H', 'T', 'H', 'T', 'H', 'H', 'H', 'T', 'T', 'T', 'H', 'T',
'H', 'H', 'T', 'H', 'T', 'T', 'H', 'H', 'H', 'H', 'T', 'T', 'H', 'H', 'H', 'T', 'H', 'H', 'T', 'T', 'T',
'T', 'T', 'H', 'T', 'H', 'T', 'H', 'T', 'T', 'T', 'H', 'H', 'H', 'T', 'H', 'H', 'T', 'H', 'H', 'H', 'H',
'T', 'T', 'H', 'T', 'H', 'H', 'H', 'H', 'T', 'H', 'H', 'T', 'T', 'H', 'T', 'H', 'T', 'T', 'T', 'H', 'T',
'H', 'H', 'H', 'T', 'H', 'T', 'H', 'H', 'T', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'T', 'H', 'T', 'T', 'H',
'H', 'H', 'H', 'H', 'H', 'H', 'T', 'T', 'T', 'H', 'H', 'H', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'H',
'T', 'H', 'T', 'T', 'T', 'H', 'H', 'H', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'H', 'H', 'H', 'T', 'H', 'H',
'H', 'T', 'T', 'T', 'T', 'T', 'H', 'T', 'H', 'H', 'H', 'T', 'H', 'T', 'T', 'T', 'T', 'H', 'H', 'H', 'T',
'T', 'T', 'T', 'T', 'H', 'H', 'H', 'T', 'H', 'H', 'H', 'H', 'H', 'T', 'T', 'T', 'H', 'H', 'H', 'H', 'H',
'H', 'H', 'H', 'T', 'T', 'T', 'H', 'H', 'H', 'T', 'T', 'T', 'H', 'T', 'H', 'H', 'T', 'T', 'T', 'T', 'T',
'H', 'T', 'H', 'H', 'T', 'H', 'T', 'H', 'T', 'T', 'T', 'H', 'T', 'T', 'H', 'H', 'H', 'H', 'T', 'T', 'T',
'T', 'H', 'H', 'H', 'H', 'T', 'T', 'H', 'H', 'T', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'T', 'H', 'T', 'H',
'H', 'H', 'T', 'H', 'T', 'T', 'H', 'T', 'T', 'H', 'T', 'H', 'T', 'H', 'H', 'T', 'T', 'T', 'H', 'T', 'T',
'T', 'T', 'H', 'T', 'H', 'H', 'T', 'T', 'H', 'H', 'H', 'H', 'T', 'H', 'T', 'T', 'H', 'T', 'H', 'H', 'T',
```

```python
both_events = sum(True for die,coin in zip(die_rolling , coin_flips) if die == 3 and coin == 'H')
both_events
```

```
829
```

```python
int(True) # 1
```

```
1
```

```python
die_coin_pair = zip(die_rolling , coin_flips)
die_coin_pair
```

```
<zip at 0x24f4b976240>
```

```python
print(tuple(die_coin_pair))
```

```
((2, 'H'), (6, 'H'), (6, 'T'), (2, 'T'), (5, 'H'), (1, 'H'), (2, 'T'), (3, 'H'), (6, 'H'), (3, 'H'), (1,
'T'), (1, 'T'), (2, 'H'), (1, 'H'), (5, 'T'), (5, 'H'), (4, 'T'), (4, 'H'), (3, 'H'), (5, 'H'), (2, 'T'),
(3, 'T'), (6, 'T'), (6, 'T'), (4, 'H'), (6, 'T'), (4, 'T'), (1, 'T'), (5, 'H'), (4, 'T'), (1, 'T'), (1,
'T'), (5, 'H'), (3, 'T'), (6, 'T'), (5, 'T'), (2, 'T'), (2, 'T'), (4, 'T'), (5, 'T'), (1, 'T'), (3, 'T'),
(3, 'T'), (2, 'T'), (6, 'H'), (1, 'H'), (1, 'H'), (6, 'T'), (3, 'T'), (6, 'T'), (1, 'H'), (1, 'H'), (1,
'T'), (4, 'H'), (4, 'T'), (3, 'H'), (1, 'H'), (6, 'H'), (5, 'T'), (2, 'T'), (1, 'T'), (1, 'H'), (4, 'T'),
(1, 'H'), (4, 'H'), (5, 'T'), (5, 'H'), (1, 'T'), (6, 'T'), (2, 'H'), (5, 'H'), (6, 'H'), (4, 'H'), (1,
'T'), (1, 'T'), (2, 'H'), (2, 'H'), (2, 'H'), (4, 'T'), (3, 'H'), (6, 'H'), (6, 'T'), (3, 'T'), (3, 'T'),
(2, 'T'), (5, 'T'), (1, 'H'), (6, 'T'), (4, 'H'), (5, 'T'), (1, 'H'), (3, 'T'), (6, 'T'), (3, 'T'), (6,
'H'), (1, 'H'), (3, 'H'), (2, 'T'), (6, 'H'), (2, 'H'), (5, 'T'), (5, 'H'), (1, 'H'), (3, 'H'), (2, 'H'),
(2, 'T'), (3, 'T'), (4, 'H'), (4, 'T'), (4, 'H'), (5, 'H'), (3, 'H'), (1, 'H'), (1, 'T'), (6, 'H'), (5,
'H'), (2, 'T'), (1, 'T'), (1, 'H'), (1, 'T'), (1, 'H'), (1, 'T'), (2, 'T'), (3, 'T'), (2, 'H'), (4, 'T'),
(2, 'H'), (3, 'H'), (5, 'H'), (2, 'T'), (3, 'H'), (1, 'T'), (6, 'H'), (6, 'H'), (5, 'T'), (1, 'H'), (6,
'T'), (6, 'T'), (5, 'T'), (5, 'T'), (5, 'H'), (4, 'T'), (4, 'T'), (6, 'H'), (3, 'T'), (6, 'T'), (4, 'H'),
(2, 'H'), (4, 'H'), (1, 'H'), (5, 'H'), (2, 'H'), (2, 'H'), (5, 'T'), (6, 'T'), (2, 'T'), (2, 'H'), (1,
'H'), (1, 'H'), (2, 'T'), (1, 'T'), (4, 'T'), (5, 'T'), (2, 'T'), (1, 'T'), (5, 'T'), (4, 'T'), (6, 'H'),
(1, 'T'), (6, 'H'), (3, 'T'), (6, 'T'), (2, 'T'), (1, 'H'), (6, 'H'), (3, 'H'), (4, 'H'), (3, 'T'), (1,
```

```python
count = 0
for die , coin in zip(die_rolling , coin_flips):
    if die == 3 and coin == 'H':
        count+=1
count
```

```
829
```

```python
probability_events = both_events / 10000
probability_events
```

```
0.0829
```

```python
1/12
```

```
0.08333333333333333
```

```python
# Dependent Events Example
# Drawing two cards from a deck without replacement creates dependency between events.
from itertools import product
deck = list(product(['1','2','3','4','5','6','7','8','9','10','J','Q','K'],
                ['Heart','Diamond','Spade','Club']))
deck
```

```
[('1', 'Heart'),
 ('1', 'Diamond'),
 ('1', 'Spade'),
 ('1', 'Club'),
 ('2', 'Heart'),
 ('2', 'Diamond'),
 ('2', 'Spade'),
 ('2', 'Club'),
 ('3', 'Heart'),
 ('3', 'Diamond'),
 ('3', 'Spade'),
```

```
 ('3', 'Heart'),
 ('3', 'Diamond'),
 ('3', 'Spade'),
 ('3', 'Club'),
 ('4', 'Heart'),
 ('4', 'Diamond'),
 ('4', 'Spade'),
 ('4', 'Club'),
 ('5', 'Heart'),
```

```
len(deck) # 52
```

```
52
```

```
random.shuffle(deck)
deck
```

```
[('6', 'Heart'),
 ('Q', 'Heart'),
 ('5', 'Diamond'),
 ('J', 'Spade'),
 ('4', 'Heart'),
 ('J', 'Diamond'),
 ('10', 'Spade'),
 ('7', 'Spade'),
 ('4', 'Spade'),
 ('9', 'Spade'),
 ('2', 'Spade'),
 ('10', 'Club'),
 ('3', 'Heart'),
 ('2', 'Diamond'),
 ('3', 'Spade'),
 ('2', 'Club'),
 ('1', 'Diamond'),
```

```
ace_king_draws = 0
trails = 10000

for _ in range(trails):
    random.shuffle(deck)
    first_card = deck.pop() # 52 cards
    second_card = deck.pop() # 51 cards
    if first_card[0] == '1' and second_card[0] == 'K':
        ace_king_draws +=1
    deck.append(first_card)
    deck.append(second_card)

probability_events = ace_king_draws / trails
probability_events
```

```
0.006
```

```
first_card
```

```
('1', 'Club')
```

```
second_card
```

```
('7', 'Club')
```